

Agile Requirements Engineering

AI Agentic Software Engineering Platform

1. Stakeholders and End Users

Engineering Team (Primary end user)

Uses the platform to manage software-engineering tasks, interact with AI agents, review AI findings and agent plans, approve agent actions, and review/merge generated pull requests.

Developer Team (Development team)

Designs, develops, tests, maintains, and improves the platform, including its AI agents, GitHub integration, memory system, and other core components.

Students (End user)

Use the platform for learning software engineering, understanding code through AI explanations, debugging, and completing academic or software-development projects.

GitHub (Third-party system)

Provides repository and development-workflow integration, including repositories, branches, issues, commits, CI results, and pull requests.

LLM API Provider (Third-party system)

Provides the language-model capabilities required for code analysis, reasoning, planning, and code generation, subject to API availability, rate limits, cost, and model capabilities.

2. Elicitation Techniques by Stakeholder

Stakeholder	Elicitation Techniques
Engineering Team	- Interviews — understand needs and workflow - Observation — see actual development activities - Prototyping — validate UI/agent workflow
Developer Team	- Interviews — identify technical requirements - Brainstorming — generate solution ideas
Students	- Questionnaires — collect broad feedback - Interviews — understand learning needs - Observation — understand actual usage - Prototyping — validate usability
GitHub	- Existing System / Documentation Analysis — understand APIs and constraints - Risk Analysis — identify integration failures and limits
LLM API Provider	- Existing System / Documentation Analysis — understand API, limits, and capabilities - Risk Analysis — handle downtime, rate limits, cost, and model failures

3. System Requirements

3.1 Functional Requirements

FR-01 — User Authentication

The system shall allow users to securely log in and access the platform according to their assigned role.

FR-02 — GitHub Repository Integration

The system shall allow the engineering team to connect a GitHub repository and access its branches, issues, commits, pull requests, and relevant CI results.

FR-03 — Repository Understanding

The system shall analyze the connected repository and identify its files, modules, dependencies, APIs, tests, and overall structure so that the AI agent can use the repository context while performing tasks.

FR-04 — Task Submission

The system shall allow a user to submit a software-development task in natural language or select an existing GitHub issue.

FR-05 — AI Task Analysis

The system shall analyze the submitted task and identify the relevant files, components, dependencies, and project context required to perform the task.

FR-06 — Agent Plan Generation

The system shall generate a step-by-step implementation plan for the submitted task and present it to the user for review before making code changes.

FR-08 — Agent Code Modification

After approval, the system shall allow the agent to modify code only within an isolated execution environment.

FR-09 — Test Generation and Execution

The system shall generate or update relevant tests for the implemented changes and execute the tests in the controlled environment.

FR-10 — Failure Analysis

When a test or CI process fails, the system shall analyze the failure, identify the related code and probable cause, and provide a suggested fix or allow the agent to attempt a fix.

FR-11 — Independent Verification

The system shall verify that the implemented solution satisfies the original task and that the relevant tests pass before preparing the changes for review.

FR-12 — Draft Pull Request Creation

The system shall create a draft pull request containing the implemented changes, test results, verification information, and a summary of the work performed.

FR-13 — Human Review and Decision

The system shall allow the engineering team to review the agent's changes and either approve them, request further changes, or reject the work.

FR-14 — Rollback and Recovery

The system shall allow the user to stop an agent task and restore the system to a previous checkpoint.

FR-15 — Project Memory

The system shall store relevant project information, decisions, completed tasks, previous agent work, failed approaches, and project rules for use in future tasks.

FR-16 — Context Retrieval

The system shall retrieve relevant stored project information when the agent starts a new task so that previous project knowledge can be considered.

FR-17 — Code Explanation

The system shall allow users to request plain-language explanations of code, errors, pull requests, test failures, and AI findings.

FR-18 — Agent Activity Logging

The system shall maintain a chronological record of significant agent activities, including task execution, code changes, test execution, corrections, and pull-request creation.

3.2 Non-Functional Requirements

NFR-01 — Performance

The system should provide repository analysis, task planning, and user-interface responses within a reasonable time under normal workload.

NFR-02 — Availability

The platform should remain available during normal working hours and should recover gracefully from temporary failures of external services such as GitHub or the LLM API.

NFR-03 — Security and Safety

The system shall protect source code, credentials, API keys, and other sensitive information from unauthorized access, while preventing AI agents from performing unauthorized or potentially harmful actions without the required permissions and human approval.

NFR-04 — Data Privacy

The system shall prevent passwords, API keys, access tokens, and other sensitive credentials from being stored in project memory.

NFR-05 — Reliability

The system shall preserve the current task state and relevant checkpoint information if an agent execution is interrupted or fails.

NFR-07 — Usability

The system shall provide a simple interface through which users can submit tasks, review agent plans, inspect results, and approve or reject actions without requiring knowledge of the internal AI implementation.

NFR-08 — Maintainability

The system shall be modular so that components such as the GitHub integration, LLM provider, memory system, and agent execution system can be modified independently.

NFR-09 — Scalability

The system should support multiple repositories and multiple agent tasks without significant degradation in performance.

NFR-10 — Auditability

The system shall maintain sufficient records of agent activities and decisions to allow a user to understand what actions were performed and what evidence was produced.

NFR-11 — Recoverability

The system shall allow interrupted or failed agent tasks to be resumed from a previously stored checkpoint where possible.

NFR-12 — Interoperability

The system shall integrate with GitHub and supported external LLM APIs through their available interfaces without requiring changes to the external systems.

3.3 Domain Requirements

DR-01 — Human Review Before Merge

The system shall not automatically merge an AI-generated change into the target branch; a human developer shall review and approve the change before merging.

DR-02 — Version-Control Based Development

All code changes generated by the system shall be associated with a version-control workflow using branches, commits, and pull requests.

DR-03 — Requirement Traceability

A completed development task should be traceable from the original requirement or issue to the changed code, relevant tests, pull request, and verification result.

DR-04 — Testing Before Review

Changes generated by the agent shall be tested before they are presented as ready for human review.

DR-05 — Project Coding Standards

The agent shall follow the engineering standards and project-specific rules defined for the repository, including required testing and coding conventions.

DR-06 — Secure Handling of Credentials

Credentials, API keys, access tokens, and other sensitive information shall not be included in persistent project memory.

DR-07 — Change Impact Awareness

Before implementing a significant change, the system should consider affected components, dependencies, APIs, and relevant tests to reduce the risk of regressions.

DR-08 — Evidence-Based Agent Decisions

The platform should provide evidence for significant agent actions so that developers can understand what was changed and why.